CIS 4190/5190 — Image Style Transfer

Complete Project Execution Plan

Penn Engineering · Spring 2026 · Team of 3

Document Sections

1. Problem Explained Simply	9. Evaluation Plan
2. Minimum Viable Project Path	10. Week-by-Week Timeline
3. A-Level Project Path	11. Team Organization
4. Data Collection Playbook	12. Questions for TA/Professor
5. Annotation & Labeling Plan	13. Curated Reading List
6. Model Roadmap (Baseline → Advanced)	14. Final Report Advice
7. If Pretrained Models Are NOT Allowed	15. Risk Management
8. If Pretrained Models ARE Allowed	FIRST 48 HOURS CHECKLIST

1. The Problem, Explained Simply

1.1 What Are We Actually Doing?

You have a photo of Locust Walk on a sunny afternoon. Your job is to produce a new image that looks like the same scene — same buildings, trees, walkways, and camera angle — but now it is raining at night. The content of the scene stays the same; only the appearance changes.

Think of it like Photoshop at the scene level. You are not adding a filter. You are asking a model to realistically re-render the entire scene under a new lighting and weather condition.

1.2 Why Is It Hard?

- The model must change everything about lighting and atmosphere while touching nothing about geometry. A building cannot move; a puddle must appear in plausible places.
- Rain, fog, and night lighting interact with surfaces in physically complex ways that simple color adjustments cannot capture.
- There is usually no perfect 'ground truth' image of the same scene under the target condition taken from exactly the same viewpoint. Your data will be imperfectly paired at best.
- Neural models love to hallucinate: generate plausible-looking but wrong content (e.g., moving a window, erasing a tree, changing a building's shape).
- Evaluation is genuinely hard: LPIPS measures perceptual quality, but a model can get a good LPIPS score while completely failing to preserve scene identity.

1.3 Paired vs. Unpaired Image Translation

These are the two fundamentally different problem settings in image-to-image translation:

Paired Translation	Unpaired Translation
Paired	Unpaired
You have image A (sunny) and image B (rainy) of the EXACT same scene from the EXACT same viewpoint.	You have many sunny images and many rainy images, but no guaranteed pairs of the same scene.
Model can learn a direct pixel-level mapping. Training signal is clear.	Model must discover what changes between conditions on its own. Much harder to train.
Requires significant real-world effort to collect. Very hard to get exact alignment.	Easier to collect (just take photos at different times), but training is less stable.
Models: pix2pix, InstructPix2Pix (supervised editing), paired diffusion fine-tuning.	Models: CycleGAN, UNIT, some diffusion-based unpaired methods.
Best path for a beginner team if you can collect even a modest paired set.	Necessary fallback if you cannot revisit locations under every condition.

> 👉 *Recommendation: Aim for loosely paired data. Collect the same viewpoints under different conditions. Images will not be perfectly pixel-aligned, but they will be strongly semantically aligned. This is the best realistic setting for your team.*

1.4 What Makes an A+ Project?

- A clear, honest comparison of at least two methods (e.g., a naive baseline vs. a pretrained editing model).
- A thoughtful data collection protocol with documented metadata, not just 'we took photos.'

- Ablation studies: showing that each component of your method matters.
- Failure case analysis: an honest discussion of where your model breaks.
- At least one genuinely interesting exploratory component (see Section 3 for ideas).
- A well-structured report that reads like a mini research paper, not a homework writeup.

2. The Minimum Viable Project Path

2.1 The Simplest Valid Baseline You Can Build in Week 1

Your Week 1 goal is to have a working end-to-end pipeline, however crude. 'Working' means: given an input image and a target condition label, your system outputs an image. It does not need to look good yet.

Baseline 0 — The Dummy Baseline

Return the input image unchanged. This is your lower bound. Compute LPIPS and condition accuracy on this. Every model you build must beat it.

Implementation time: 30 minutes. Risk: none.

Baseline 1 — Image Lookup / Color Jitter

Given an input sunny daytime image, find an image in your dataset that is labeled 'night + rainy' from the same location (if it exists) and return it. If no pair exists, apply a simple color transform (darken the image for night, add a blue-gray tint for rain).

This is a real, submittable baseline. It is naive but honest, and it establishes both metrics against which future models are measured.

2.2 What Data to Use First

Do NOT wait for a large dataset. Start with a pilot of 3 locations × 3 conditions = 9 groups of images, roughly 3-5 images per group. That is about 30-45 images total. This is enough to build and test your pipeline.

- Location 1: Locust Walk (wide, recognizable, good lighting variation)
- Location 2: Fisher Fine Arts Library front steps (distinctive architecture)
- Location 3: College Green / a quad area (open sky, good for weather changes)

Collect these locations in at least 2 conditions in week 1. You will expand later.

2.3 The First Model to Try

> ⭐ *Best first model for beginners: InstructPix2Pix (if permitted) OR a color-transfer approach with a CLIP-guided classifier as condition checker.*

If pretrained editing models are allowed (see Section 8):

- Use InstructPix2Pix from Hugging Face Diffusers. Pass your image + a text instruction like 'make this scene look like a rainy night.' Run inference. Measure LPIPS and condition accuracy.
- Implementation time: ~4 hours for a beginner who follows the Diffusers tutorial carefully.

If pretrained editing models are NOT allowed (see Section 7):

- Use a pretrained CLIP model only to build a condition classifier, and start with a color transform baseline while you set up a CycleGAN or pix2pix training loop.

2.4 The Fallback Plan if Your First Model Fails

Problem	Fallback Solution
If InstructPix2Pix outputs garbage on campus scenes	Switch to img2img with Stable Diffusion + a stronger text prompt. Add ControlNet for structure preservation.
If CycleGAN training diverges	Reduce learning rate to 1e-5, add more augmentation, reduce the discriminator capacity.

If you run out of Colab GPU	Use Google Colab Pro (cheap), limit batch size to 1, use gradient checkpointing, or switch to a lighter model (pix2pix on 128×128 crops).
If you cannot collect data fast enough	Use the provided sample data + publicly available seasonal/weather image datasets to bootstrap training, then fine-tune on your campus data.
If LPIPS is terrible across all methods	Report it honestly. Show improvement trend across model iterations. The grade is about your process, not a magic number.

3. The A-Level Project Path

3.1 Stronger Methods to Consider After the Baseline

Once your baseline is running, the goal is to systematically improve it and document what works. Here is the recommended upgrade sequence:

Level	Method	What It Adds
Level	Method	What it adds
B0	Color transform / image lookup	Baseline — sets the floor
B1	InstructPix2Pix (zero-shot)	Pretrained knowledge, no fine-tuning needed
B2	SD img2img + ControlNet (Canny/Depth)	Structure preservation via explicit conditioning
A-	LoRA fine-tuning on campus data	Domain adaptation to Penn campus appearance
A	ControlNet + LoRA + condition prompt engineering	Full control over structure + content + condition
A+	Any of the above + a custom exploratory angle	Novel contribution: segmentation masks, retrieval, multi-stage

3.2 Ablation Studies That Make Your Project Strong

An ablation study removes one component at a time and shows that each piece matters. For this project, strong ablations include:

- With vs. without ControlNet: does structure conditioning help preserve scene identity?
- With vs. without domain fine-tuning: does LoRA on campus data improve LPIPS vs. the off-the-shelf model?
- Text prompt comparison: 'make it rainy' vs. 'a photo of Locust Walk at Penn at night in heavy rain with wet pavement' — does prompt quality matter?
- Condition granularity: binary condition (day/night) vs. fine-grained (daytime/sunset/night) — does it affect accuracy?
- Data size ablation: train with 20, 50, 100 images — where does performance saturate?

3.3 Exploratory Directions with Specific Advice

A. Better Condition Encoding

Instead of free-text prompts, encode conditions as structured embeddings. Train a small condition encoder (e.g., embed time-of-day and weather as one-hot or learned vectors) and inject it into the model. Compare vs. text prompts only. This is a clean, well-scoped exploratory contribution.

B. Retrieval-Augmented Generation

Build a CLIP-based image retrieval system. Given an input scene, retrieve the closest matching reference image in your dataset that shows the target condition. Feed this reference to the generative model as additional context. This is a creative direction that does not require heavy training.

C. LoRA / Adapter Fine-Tuning

LoRA (Low-Rank Adaptation) lets you fine-tune a large diffusion model with as few as 30-50 images using only a small fraction of the model parameters. Use diffusers + peft library. Train one LoRA adapter per condition pair. This is feasible on free Colab and is a concrete, reportable contribution.

D. ControlNet-Style Conditioning

ControlNet takes a structural signal (Canny edges, depth map, semantic segmentation) as an explicit conditioning input alongside the text prompt. Use a pretrained ControlNet checkpoint. Your exploratory angle: compare which structural signal (Canny vs. depth vs. segmentation) best preserves scene identity on campus scenes.

E. Segmentation-Guided Editing

Run a semantic segmentation model (e.g., SegFormer) on each scene. Use the sky mask to edit only sky pixels, keep building pixels fixed. This is a clean, interpretable contribution that avoids hallucination on structural elements.

F. Multi-Stage Pipelines

Stage 1: predict the sky condition change (e.g., replace sky using inpainting). Stage 2: propagate lighting changes to the rest of the scene. This compositional approach is easy to analyze and makes for a clean ablation.

4. Data Collection Playbook

4.1 How Much Data to Collect

Rule of thumb: 5 locations × 5 viewpoints × 3-4 conditions = 75-100 base images. This is your minimum viable dataset. Aim for 150-250 total images before the final submission.

Parameter	Target Value
Locations on campus	5-8 (start with 5: Locust Walk, Fisher-Bennett, College Green, Hill House area, Towne Building)
Viewpoints per location	3-5 (standing, looking in different cardinal directions from the same spot)
Conditions per viewpoint	At least 2, ideally 3 (sunny/daytime, overcast/cloudy, and one of: night, rainy, sunset)
Images per viewpoint per condition	3-5 (slight angle variations are fine; this gives you robustness)
Total target	150-250 images minimum; 400+ is great if you have time

4.2 What to Do BEFORE Collecting a Lot of Data

1. Build the pipeline first. Collect 20 images from 2 locations, run your baseline model, and confirm the pipeline works end-to-end.
2. Define your naming convention and metadata CSV format (see 4.4) before your first collection session so nobody has to rename anything later.
3. Take a test 'pilot batch' at one location in one condition. Extract EXIF data, verify GPS is recorded, check image sharpness, confirm orientation is consistent.
4. Review the pilot images together as a team. Agree on what 'good enough' looks like before scaling up.

4.3 Paired vs. Unpaired Data — What to Prioritize

Prioritize paired data. Revisiting the same location and viewpoint under different conditions is far more valuable than collecting many unrelated scenes. The LPIPS metric requires ground-truth target images, so pairs are necessary for evaluation.

- Week 1-2: collect your 5 pilot locations in at least 2 conditions each.
- Week 2-3: revisit and collect 1-2 additional conditions at each location.
- Week 3-4: add 2-3 new locations to expand scene diversity.

4.4 Folder Structure and Filenames

Folder: data/ raw/ loc001_locut_walk/ vp01_north/ sunny_day/ loc001_vp01_sunny_day_001.jpg loc001_vp01_sunny_day_002.jpg night_clear/ loc001_vp01_night_clear_001.jpg vp02_east/ ... metadata.csv splits/ train.csv val.csv test.csv

4.5 Metadata CSV Columns

Column	Example Value	Notes
filename	loc001_vp01_sunny_day_001.jpg	Unique image filename

location_id	loc001	Standardized location code
location_name	Locust Walk (north end)	Human-readable name
viewpoint_id	vp01	Viewpoint code within location
time_of_day	daytime	One of: daytime, sunset, night
weather	sunny	One of: sunny, cloudy, rainy
condition_label	daytime_sunny	Combined condition: time_weather
collector	teammate_a	Who collected this image
collection_date	2026-03-28	ISO 8601 date
approx_time	14:30	Time of day when photo was taken
gps_lat	39.9522	From EXIF if available
gps_lon	-75.1932	From EXIF if available
split	train	train / val / test
notes		Any issues: blurry, occluded, etc.

4.6 Common Beginner Data Collection Mistakes

- Zoom: Never zoom in or out between shots of the same viewpoint. Lock the focal length.
- Phone orientation: Always hold the phone in the same orientation (landscape recommended for wider scenes). Portrait is fine if you are consistent.
- Weather mislabeling: 'Cloudy' and 'overcast' mean the same thing. Define labels before collecting (see Section 5).
- Too many locations, not enough conditions: 20 locations with only sunny images are nearly useless. 5 locations with 4 conditions each is far better.
- Forgetting EXIF: Enable GPS tagging before your first collection trip. Test it immediately.
- No metadata: Do not rely on memory. Fill the CSV during or immediately after collection.
- Privacy: Avoid images dominated by clearly identifiable faces. Blur or exclude them.
- Motion blur: In low light (night, rain), use both hands and brace against a surface.

4.7 How to Avoid Data Leakage

Split data by location+viewpoint groups, not by individual images. If a viewpoint appears in training, ALL of its images (across conditions) should stay in training. Never split such that the same viewpoint appears in both train and test.

- Assign location+viewpoint groups to splits first, then filter images by group.
- Keep test set in a separate folder, locked, until final evaluation.

5. Annotation and Labeling Plan

5.1 Exact Label Definitions

Use these exact definitions. Write them in a shared document and review them together before your first collection session.

Time of Day (pick exactly one)

Label	Definition
daytime	Sun is clearly above the horizon. Shadows are well-defined. Sky is blue or overcast but bright. Typically 7am–5pm.
sunset	Sun is at or just below the horizon. Sky is orange, pink, or deep red. Golden-hour lighting. Roughly 5pm–7pm depending on season.
night	It is dark. Artificial lighting (streetlights, building lights) is the primary light source. Sky is dark. After 7pm or whenever it is clearly dark.

Weather (pick exactly one)

Label	Definition
sunny	No clouds or very light cloud cover. Direct sunlight is visible. Shadows are sharp.
cloudy	Overcast sky. No direct sunlight. Sky is gray or white. No precipitation.
rainy	Active rain or wet pavement/surfaces. Raindrops visible or ground is clearly wet.

5.2 Labeling Checklist

Use this checklist for every image during collection. □ Is the EXIF GPS data recorded? (check on phone before moving) □ Has the filename been assigned per convention? □ Has the metadata CSV row been filled in? □ Is the time_of_day label correctly assigned (check definitions above)? □ Is the weather label correctly assigned (check definitions above)? □ Is the image sharp (not motion-blurred)? □ Does the image show the intended scene (no obstruction, no major occlusion)? □ Does the image avoid faces or privacy-sensitive content? □ Is the phone held in the correct orientation? □ If this is a revisit, does the framing match the previous image at this viewpoint?

5.3 Resolving Label Disagreements

When teammates disagree on a label (e.g., is this 'sunset' or 'daytime'?):

5. Check the definition. If the image clearly falls within one definition, use that label.
6. If still ambiguous, the image goes to a 'review' pile. All three teammates vote. Majority wins.
7. If the image is genuinely ambiguous (e.g., barely-onset golden hour), consider excluding it from the paired validation set, where precise labels matter most.

6. Model Roadmap — From Easiest to Hardest

Baseline 0 — Identity / Dummy Baseline

Attribute	Details
Why try it	Establishes the floor. Lets you verify your evaluation code is working correctly.
Data needed	Any images. Just return the input unchanged.
Compute needed	Zero. CPU-only, instant.
Risk	None.
Success looks like	Pipeline runs. You have LPIPS and condition accuracy numbers for the identity case.
Implementation time	30 minutes.
What could go wrong	If LPIPS = 0.0 for identity, your LPIPS computation is wrong (it should be nonzero vs. a different-condition ground truth).

Baseline 1 — Color Transform / Image Lookup

Attribute	Details
Why try it	Simple, interpretable, and surprisingly competitive. Forces you to understand what a 'good' output even looks like.
Data needed	A small set of labeled images.
Compute needed	CPU-only.
Risk	Low. The output looks fake, but that's expected at this stage.
Success looks like	Output images are plausibly darker for night, bluer-gray for rain. LPIPS is better than random noise.
Implementation time	2-4 hours.
What could go wrong	Color transforms may introduce harsh artifacts. Use histogram matching or LUT-based transforms for smoother results.

Baseline 2 — InstructPix2Pix (Conditional on Permission)

Attribute	Details
Why try it	Hugging Face provides a one-liner inference API. You get high-quality zero-shot results in hours with minimal code.
Data needed	Your campus images. No fine-tuning needed for zero-shot.
Compute needed	Free Colab T4 GPU is sufficient for inference on 256×256 images.
Risk	Medium. Results may not preserve campus-specific identity well without fine-tuning.
Success looks like	Generated images look realistic and match the target condition. Condition accuracy > 70%.

Implementation time	4-8 hours including setup, debugging, and running full evaluation.
What could go wrong	Model hallucinates campus-specific features. Mitigation: add ControlNet (see Strong Model below).

Strong Model — ControlNet + Domain Adaptation (LoRA)

Attribute	Details
Why try it	ControlNet provides explicit structural conditioning (Canny edges or depth) that drastically reduces hallucination. LoRA adapts the model to your specific campus dataset cheaply.
Data needed	30-100 paired or unpaired campus images for LoRA training. Pretrained ControlNet checkpoint (freely available).
Compute needed	LoRA training on Colab Pro (~$10/month) is feasible. Expect 1-3 hours per fine-tuning run.
Risk	Medium-high. LoRA fine-tuning can overfit on tiny datasets. Use strong regularization.
Success looks like	LPIPS improves vs. zero-shot by 10-20%. Structural elements (buildings, walkways) are stable across conditions.
Implementation time	1-2 weeks including debugging.
What could go wrong	Overfitting. Mode collapse. Fix: use more diverse training data, reduce LoRA rank, add dropout.

7. If Pretrained Image-Editing Models Are NOT Allowed

> ⚠️ *Wait for TA/Professor clarification before assuming this branch. Ask the exact questions in Section 12. Only fall back to this plan if you get a clear 'no' on InstructPix2Pix and similar models.*

7.1 Your New Primary Strategy: CycleGAN or pix2pix

If you cannot use large pretrained editing models (InstructPix2Pix, SD-based editing), your primary model family is conditional image translation trained from scratch on your campus data.

- pix2pix (if you have paired data): Given aligned pairs (same scene, different conditions), train a U-Net generator with a PatchGAN discriminator. This is feasible on Colab in 3-6 hours of training.
- CycleGAN (if unpaired): Two generators, two discriminators, cycle consistency loss. More complex to train, but does not require exact paired data.

7.2 How to Make It Feasible

8. Reduce image resolution to 128×128 or 256×256. This cuts training time by 4-16x.
9. Train one model per condition pair (e.g., one model for day→night, one for sunny→rainy) rather than a single multi-condition model. This simplifies each training task.
10. Use a pretrained VGG or ResNet as a frozen feature extractor for perceptual loss only. Feature extractors for loss computation are generally considered tools, not generative models — confirm this with the professor.
11. Limit to 3 conditions: daytime_sunny, night_clear, daytime_rainy. Three separate GAN models.

7.3 What Alternate Technical Contribution You Can Make

- Dataset: The campus dataset you collect IS a novel contribution. Document it carefully and make it the centerpiece of the paper.
- Analysis: Train CycleGAN and pix2pix and compare them rigorously on your campus data. This is a clean, tractable comparison that the class has not seen before on this specific domain.
- Teaching: Write a detailed blog post or Jupyter notebook explaining CycleGAN with your campus examples as illustrations. This counts as an exploratory component.

7.4 What to Say to the Professor

'We plan to use a pretrained VGG-16 only as a frozen feature extractor for computing perceptual loss during GAN training, not as a generative editing model. We will train our generator and discriminator from scratch on our campus dataset. Is this approach acceptable under the project constraints?'

8. If Pretrained Image-Editing Models ARE Allowed

8.1 Which Category of Pretrained Models to Consider First

Category	Recommended Option	Why It Fits
Model Category	Recommended Option	Why It Fits
Instruction-based editors	InstructPix2Pix (timbrooks/instruct-pix2pix on HF)	Simplest API. Text instruction → edited image. Beginner-friendly.
Diffusion img2img	Stable Diffusion 1.5 img2img + ControlNet	More control over output. Better structure preservation.
Inpainting + sky replacement	Stable Diffusion Inpainting + SAM sky mask	Conservative approach: only edit the sky and lighting.
CLIP-guided GAN	StyleGAN + CLIP direction methods	More experimental, harder to get working on campus scenes.
Video/temporal	Any video diffusion model	Overkill for this project. Avoid.

8.2 How to Choose Between Them

Default recommendation: Start with InstructPix2Pix. It requires the least code, works in inference-only mode, and gives you a strong baseline quickly. Then add ControlNet on top if scene preservation is insufficient.

8.3 What to Fine-Tune vs. Freeze

Component	Details
Freeze (do not train)	The UNet backbone of Stable Diffusion. The CLIP text encoder. The VAE decoder.
Fine-tune / adapt	LoRA layers injected into the UNet attention layers (rank 4-16 is fine for 100 images). Optionally the text token embeddings (Textual Inversion).
Why this split	Training the full UNet on 100 images will catastrophically overfit. LoRA uses ~0.1% of parameters and regularizes itself.

8.4 Dataset Size Guidance

Approach	Data Needed
Zero-shot (no fine-tuning)	0 images needed. Just run inference with InstructPix2Pix.
Textual Inversion	5-20 images per concept (e.g., 'Penn campus' token).
LoRA fine-tuning	30-100 images per condition pair. More is better but diminishing returns after ~150.
Full fine-tuning	Do not attempt with less than 10k images. Not recommended for this project.

8.5 How to Avoid Overfitting on Tiny Campus Data

- Use LoRA with rank ≤ 8 for very small datasets (< 50 images).
- Add augmentation: horizontal flips, slight color jitter, random crops.
- Use a validation set of 10-15 held-out images. Stop training when validation LPIPS stops improving.
- Mix your campus images with publicly available weather/lighting images during training to provide diversity.

9. Evaluation Plan

9.1 Official Metrics

The course will evaluate your submission on two primary metrics on a closed test set:

- LPIPS (↓, lower is better): Perceptual similarity between your generated image and the ground-truth image of the same scene under the target condition. Uses deep VGG features.
- Condition Accuracy (↑, higher is better): A fixed classifier checks whether your generated image actually looks like the target condition (daytime/sunset/night × sunny/cloudy/rainy).

9.2 How to Build Your Validation Set

12. Reserve 20% of your location+viewpoint groups as a validation set before any training. These locations should NOT be visited for training data.
13. Within each validation location+viewpoint group, have at least 2 condition pairs (source → target).
14. Lock the test split. Never train on it, never tune hyperparameters using it.
15. Report validation numbers for model selection, and test numbers only for your final reported result.

9.3 How to Measure Content Preservation

LPIPS measures perceptual quality vs. ground truth, but you should also report:

- SSIM (Structural Similarity Index): measures preservation of edges, textures, and spatial structure.
- Semantic segmentation consistency: run a segmentation model on the input and output. Check that the class distribution (buildings, sky, trees, walkways) is similar.
- DINO feature similarity: compare ViT-DINO features of input and output. High similarity = good content preservation.

9.4 How to Measure Condition Accuracy

Build or use a pretrained weather/time-of-day classifier:

- Option A (easiest): Use CLIP zero-shot classification. Prompt: 'a photo taken during [condition]' and compare cosine similarity.
- Option B: Fine-tune a ResNet-18 or ViT-S on publicly available time/weather labeled datasets. Apply to your generated images.
- Option C: Use the course's fixed classifier when provided.

9.5 How to Create Comparison Tables and Figures

Figure/Table Type	How to Create It
Quantitative table	One row per model variant. Columns: Method | LPIPS ↓ | Condition Acc ↑ | SSIM ↑ | Notes. Include confidence intervals or std dev if you run multiple seeds.
Qualitative figure	3×N grid: Row 1 = input, Row 2 = your model output, Row 3 = ground truth (if available). Show 3-5 examples covering different conditions.
Ablation table	Same format as the quantitative table, but only your method variants. Label each row by what is added/removed.
Failure case figure	3 examples where your model clearly fails. Label what went wrong.

Condition accuracy confusion matrix	Shows which condition transitions are hardest for your model.

9.6 How to Do Ablations

Plan your ablations before you start training, not after. Define them as your baseline + one change at a time:

- Ablation 1: No ControlNet (just text prompt)
- Ablation 2: ControlNet with Canny vs. ControlNet with Depth
- Ablation 3: No LoRA fine-tuning (zero-shot only)
- Ablation 4: LoRA rank 4 vs. rank 16
- Ablation 5: Short prompt vs. detailed prompt

Each ablation should run on the same validation set. Report all results in a single table.

10. Week-by-Week Timeline

Deadline: May 6, 2026. You have approximately 6 weeks from project selection. Treat the first 4 weeks as your sprint, the last 2 as polish and report writing.

Week 1 (March 25 – April 1): Setup and Pilot

Task Area	B	C
Teammate A (Data Lead)	Teammate B (Model Lead)	Teammate C (Eval/Report Lead)
Define 5 pilot locations. Create folder structure and metadata CSV template.	Set up GitHub repo, Google Colab environment, install diffusers, torch, lpips libraries.	Read project spec carefully. Write evaluation code skeleton: load image pairs, compute LPIPS and condition accuracy.
Collect 30-50 pilot images from 3 locations in 2 conditions each.	Get InstructPix2Pix running on one test image (even if result is bad).	Build a simple CLIP-based condition classifier. Test it on 10 images.
Fill metadata CSV for all collected images. Verify EXIF GPS data is present.	Run Baseline 0 (identity) and record LPIPS score.	Set up a results spreadsheet: model name, LPIPS, condition accuracy, notes.
DONE: 30-50 images with complete metadata in GitHub.	DONE: Pipeline runs end-to-end. Baseline 0 score is recorded.	DONE: Evaluation code works on pilot data.

Week 2 (April 1 – 8): Baseline Models

Task Area	B	C
Teammate A	Teammate B	Teammate C
Scale data collection to all 5 locations, add a 3rd condition where possible. Target 80-120 images.	Implement Baseline 1 (color transform). Run on validation set. Record scores.	Analyze Baseline 0 and Baseline 1 qualitatively. Write notes on failure modes.
Begin metadata review: check for mislabels, blurry images. Clean the dataset.	Set up InstructPix2Pix inference. Run on all validation images with naive prompts.	Compute LPIPS and condition accuracy for both baselines. Add to results spreadsheet.
DONE: 100+ images, cleaned and labeled.	DONE: InstructPix2Pix results on validation set.	DONE: First version of results table with 2 baselines.

Week 3 (April 8 – 15): Strong Model Development

Task Area	B	C
Teammate A	Teammate B	Teammate C
Add 2-3 new locations. Total: 7-8 locations, 150+ images.	Add ControlNet to InstructPix2Pix. Experiment with Canny vs. Depth signal.	Begin report draft: Introduction, Data Collection section, Baseline Results.

Create train/val/test splits. Lock test set.	Experiment with prompt engineering. Compare 3 different prompt formulations.	Run ablation: ControlNet vs. no ControlNet. Record results.
DONE: Full dataset locked and split.	DONE: Best prompt + ControlNet config found.	DONE: 2-page report draft, ablation table started.

Week 4 (April 15 – 22): Fine-Tuning and Exploratory Component

Task Area	B	C
Teammate A	Teammate B	Teammate C
Help with LoRA training data preparation. Ensure balanced condition representation.	Begin LoRA fine-tuning on campus data. Track training loss and val LPIPS.	Work on exploratory component (segmentation-guided editing OR retrieval system). Document it.
Final data audit: remove duplicates, fix any mislabels.	Run full ablation suite (prompts, ControlNet signals, LoRA rank).	Write Model Design section and Evaluation section of the report.
DONE: Final dataset is clean and documented.	DONE: Best model selected. All ablations recorded.	DONE: 4-page report draft with figures and tables.

Final Stretch (April 22 – May 6): Polish and Submit

- Generate final qualitative examples: 5 input/output pairs per condition, including failure cases.
- Run final model on the locked test set. Record and report final numbers.
- Write Limitations and Future Work section (be honest — it helps your grade).
- Write the Exploratory Component section with supporting evidence.
- All teammates review the report. Aim for clear, concise writing — 5 pages fills fast.
- Package dataset, model weights/checkpoints, and code. Write reproduction instructions.
- Submit on Gradescope by May 6, 2026.

11. Team Organization

11.1 Role Assignments

Role	Responsibilities
Data Lead (Teammate A)	Owns data collection protocol, metadata CSV, folder structure, and data cleaning. First point of contact for any data quality question. Does NOT work in isolation — all teammates collect data together on scheduled trips.
Model Lead (Teammate B)	Owns the training pipeline, experiment configuration, model checkpoints, and Colab notebooks. Responsible for making sure experiments are reproducible.
Eval/Report Lead (Teammate C)	Owns the evaluation code, results spreadsheet, figure generation, and report writing. Reviews all quantitative claims before they go in the report.

⚠️ *Important: Roles are for accountability, not isolation. Every teammate must understand the full pipeline. Schedule 1-2 cross-training sessions where each person explains their component to the other two.*

11.2 Git Workflow

- One shared GitHub repository. Create it in Week 1.
- Branch naming: feature/teammate-name-description (e.g., feature/alice-controlnet-experiments)
- Pull request before merging any code to main. At least one teammate reviews.
- Never commit raw images to Git. Use a shared Google Drive folder for the dataset and reference it in the README.
- Commit experiment configs and results JSON files. Do not rely on Colab logs alone.

11.3 Experiment Tracking

Use a simple Google Sheets file as your experiment log. Columns: experiment_id, date, model, config_notes, LPIPS, condition_accuracy, SSIM, runtime, notes. This is infinitely simpler than setting up W&B for a 6-week project and will serve you well enough.

11.4 Weekly Meeting Cadence

- Weekly sync (30 min): What did each person finish? What is blocked? What is the plan for the next 7 days?
- Mid-week async update: Post a 2-sentence status update in your group chat on Wednesday.
- Before any major decision (model choice, ablation plan), discuss synchronously, not over text.

12. Questions to Ask the TA / Professor

> *Ask these questions as early as possible — ideally in the first week. Some answers will significantly change your strategy. Post on Ed Discussion so the entire class benefits from the answer.*

About Pretrained Models

- **'Are we allowed to use pretrained image-editing models such as InstructPix2Pix or Stable Diffusion-based editing pipelines for our Image Style Transfer project?'**
- 'If pretrained editing models are allowed, do we need to fine-tune them on our campus data, or is zero-shot inference also acceptable?'
- 'Are we allowed to use pretrained models only as components (e.g., a frozen VGG for perceptual loss, or CLIP for condition classification), even if the generative model must be trained from scratch?'
- 'Is ControlNet (which uses a pretrained Stable Diffusion backbone with a fine-tuned control adapter) considered a pretrained editing model for the purposes of this project?'

About Data

- 'Are we allowed to use publicly available weather or time-of-day image datasets (e.g., BDD100K, KITTI, seasonal outdoor datasets) to supplement our campus collection for training, as long as our evaluation is on campus images?'
- **'Are we allowed to use synthetic data generated by other models (e.g., Stable Diffusion-generated rainy campus scenes) as training data?'**
- 'Will the closed test set be drawn exclusively from campus images you have collected, or could it include images from teams' submitted datasets?'

About Evaluation

- 'What condition classifier will be used for the official Condition Accuracy evaluation? Can you share the model or its performance characteristics so we can calibrate our internal evaluations?'
- 'For LPIPS, will ground-truth target images be from the same viewpoint as the source image, or approximately the same viewpoint?'
- 'Is there a minimum threshold of performance (LPIPS or Condition Accuracy) that is required to receive the 15% benchmark grade, similar to the other project tracks?'

About Scope

- 'For the exploratory component, what counts as a strong direction for Image Style Transfer? For example, does implementing LoRA fine-tuning on top of a baseline model count as a technique contribution, or is that considered part of the basic component?'
- 'Is comparing three different model architectures (e.g., color transform, CycleGAN, InstructPix2Pix) a valid comparison-type exploratory component?'
- 'Would a detailed teaching resource (e.g., an annotated Jupyter notebook explaining diffusion-based image editing for campus scenes) count as a valid exploratory component?'

13. Curated Reading List

Start Here (Read Before Anything Else)

Resource	Why It Matters & What to Read
The project specification PDF (read it 3 times)	Understand exactly what is required vs. optional. Note the evaluation metrics precisely.
'Image-to-Image Translation with Conditional Adversarial Networks' (Isola et al., 2017 — pix2pix paper)	This is the foundational paper for paired image translation. Understanding pix2pix gives you the conceptual vocabulary for everything that follows. Read the introduction and method sections.
'Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks' (Zhu et al., 2017 — CycleGAN paper)	The standard approach for unpaired translation. Read alongside pix2pix to understand the paired vs. unpaired distinction concretely.
Hugging Face Diffusers 'Getting Started' tutorial	Practical. Shows you how to run Stable Diffusion inference in Python in 10 lines of code. Do this before reading any more theory.
'LPIPS: The Unreasonable Effectiveness of Deep Features as a Perceptual Metric' (Zhang et al., 2018)	Short paper explaining your primary evaluation metric. Read the abstract and Figure 1. Understand what LPIPS measures and what it does not.

Next Level (Read in Week 2-3)

Resource	Why It Matters & What to Read
'InstructPix2Pix: Learning to Follow Image Editing Instructions' (Brooks et al., 2023)	The paper behind InstructPix2Pix. Read the method section to understand how text-conditioned image editing works.
'Adding Conditional Control to Text-to-Image Diffusion Models' (Zhang et al., 2023 — ControlNet paper)	Explains how ControlNet conditions a diffusion model on structural signals. Read the method section. This is directly relevant if you pursue the strong model path.
'LoRA: Low-Rank Adaptation of Large Language Models' (Hu et al., 2022)	Originally for LLMs, but identical technique is used for diffusion. Read to understand why LoRA is efficient and how rank affects capacity.
'Denoising Diffusion Probabilistic Models' (Ho et al., 2020)	The foundational diffusion paper. You do not need to implement it, but reading sections 1-3 will demystify how diffusion models work.
'High-Resolution Image Synthesis with Latent Diffusion Models' (Rombach et al., 2022 — Stable Diffusion paper)	Explains the architecture behind Stable Diffusion. Focus on the latent encoding + conditioning mechanism.

Implementation Docs and Tutorials

Resource	What It Teaches
Hugging Face Diffusers documentation (huggingface.co/docs/diffusers)	Your primary implementation reference. Has guides on InstructPix2Pix, ControlNet, LoRA, img2img. Bookmark this.
diffusers/examples/instruct_pix2pix on GitHub	The official fine-tuning script for InstructPix2Pix. Copy and modify it.
diffusers/examples/controlnet on GitHub	Official ControlNet training and inference examples.
peft library documentation (Hugging Face PEFT)	For LoRA fine-tuning of diffusion models. Has a specific tutorial for Stable Diffusion LoRA.
PyTorch Image Quality library (piq) on GitHub	Easy-to-use SSIM, LPIPS, and PSNR computation. Use this for your evaluation code.
'Practical Deep Learning' fast.ai course (free, online)	If any teammate has no PyTorch experience, start here. Chapter 1 covers enough to get started.

14. Final Report Advice

> *The report is 5 pages. This is tight. Every sentence must earn its place. Write in the style of a short NeurIPS/CVPR paper — clear, precise, past tense for experiments, present tense for claims.*

14.1 Suggested Section Structure

Section	Content
Introduction (0.5 page)	What is the problem? Why is it hard? What did you do? What were your main findings? One paragraph each.
Data Collection (0.75 page)	How many images, how many locations, what conditions. Your collection protocol. Key dataset statistics table. How you split train/val/test and why.
Method (1.25 pages)	Describe each model you tried in order. Include architecture diagrams if space allows. Describe your condition representation. Describe the fine-tuning or training procedure.
Experiments (1.5 pages)	Your main results table (all models, all metrics). At least one ablation table. 1 qualitative comparison figure. 1 failure case figure.
Exploratory Component (0.5 page)	What you tried beyond the baseline. What it showed. Be specific.
Conclusion and Limitations (0.5 page)	What worked. What did not. What you would do next. Be honest about limitations.

14.2 Required Figures

- Figure 1: 3-column qualitative comparison grid (input | your model output | ground truth) for 3 different conditions.
- Figure 2: Ablation results — a bar chart of LPIPS and condition accuracy for each ablation variant.
- Figure 3: At least 2 failure cases with captions explaining what went wrong and why.
- Figure 4 (optional but strong): A diagram of your model pipeline.

14.3 Required Tables

- Table 1: Main results — all model variants, all metrics, formatted cleanly.
- Table 2: Dataset statistics — number of images per condition, per split.
- Table 3 (if space): Ablation study results.

14.4 How to Write the Limitations Honestly

Do not hide your limitations. Reviewers (TAs) reward honest analysis. Good framing: 'Our method preserves building facades well but struggles with night scenes that require complex lighting reconstruction (see Figure 3). This is likely due to the limited number of night images in our training set (N=23). A natural extension would be...'

14.5 Making the Project Sound Thoughtful and Rigorous

- Every design choice should have a stated reason. Not 'we used ControlNet' but 'we used ControlNet because preliminary experiments showed that text-only conditioning failed to preserve the building facades (LPIPS=0.42 vs. 0.31 with ControlNet).'

- Report variance. If you ran only one seed, say so. If you ran three seeds, report mean ± std.
- Cite the papers whose models you used. This is mandatory.
- Name your dataset and give it a version number (e.g., 'Penn Campus Weather Dataset v1').

15. Risk Management

Risk	Probability	Mitigation Strategy
Risk	Probability	Mitigation
Not enough data	Medium	Start collection Week 1. Set a hard floor of 80 images before any model training. Use public datasets to supplement if needed (ask TA first).
Poor weather coverage (no rainy images)	High for rainy/night	Check the weather forecast and schedule collection trips specifically when rain or sunset is expected. Locust Walk at night is always available. Night is easy to get.
Misaligned paired images	Medium-High	Use ControlNet to handle imperfect alignment structurally. Report this as a known limitation. Use DINO similarity to filter out badly misaligned pairs.
Model hallucinations (changing buildings)	High for diffusion models	Add ControlNet (Canny edges) to anchor structure. Use segmentation to compare structural elements between input and output.
Lack of compute (Colab limits)	Medium	Use Colab Pro ($10/month) for the training runs. For inference-heavy experiments, batch your images overnight. Consider using 256x256 resolution during development.
Overfitting on tiny dataset	Medium	Use LoRA with rank ≤ 8. Add augmentation. Use a held-out validation set. Stop at the checkpoint with best validation LPIPS.
Poor or inconsistent labels	Medium	Use the definitions from Section 5. Do a joint label review session as a team in Week 1. Never label alone without reviewing against the definitions.
Slow implementation (weeks 1-2 lost)	Medium	Use the InstructPix2Pix Hugging Face demo as your starting point — it is 15 lines of code. Do not write a model from scratch until you have a working pipeline.
Unclear TA/Professor guidance	Low but impactful	Ask your questions in Week 1 on Ed Discussion. Have a fallback plan ready (Section 7) so you can keep working while waiting for answers.
Teammate availability mismatch	Medium	Set a fixed weekly sync at the start. Rotate data collection duties so nobody is a single point of failure. Document everything in the shared repo.

⚡ First 48 Hours — Exact Checklist

> *Do these in order. Do not skip ahead. The goal is to have a working skeleton pipeline and a small real dataset within 48 hours. Everything else follows from this.*

Hour 0-4: Setup (All Teammates Together)

- [x] Create a shared GitHub repository. Add all teammates as collaborators.
- [ ] Create a shared Google Drive folder for images. Share with all teammates.
- [ ] Copy the metadata CSV template from Section 4.5 into the Drive folder.
- [ ] Create a shared Google Sheet called 'Experiment Log' with the columns from Section 11.3.
- [ ] Post your questions from Section 12 on Ed Discussion. Do this today.
- [ ] Install required libraries on Google Colab: pip install diffusers transformers accelerate peft lpips torch torchvision

Hour 4-8: Pilot Data Collection (All Teammates, or at least 2)

- [ ] Choose your first 3 locations (we suggest Locust Walk, Fisher-Bennett, and College Green).
- [ ] Enable GPS/EXIF on all phones before leaving. Test with one photo and verify GPS is stored.
- [ ] At each location: stand in 2-3 viewpoints. Photograph in the current condition. Take 3-5 images per viewpoint.
- [ ] Target: collect 30-45 images today across 3 locations.
- [ ] Note the approximate time and current weather condition for each group of photos.

Hour 8-16: Data Ingestion and Baseline

- [ ] Upload images to Google Drive. Rename following the convention: loc001_vp01_sunny_day_001.jpg
- [ ] Fill in the metadata CSV for all collected images.
- [ ] Write or copy a script to extract EXIF GPS data and verify it is present.
- [ ] Write Baseline 0: a Python function that takes an image path + condition label and returns the input image unchanged.
- [ ] Write the evaluation function: compute LPIPS between the output and a ground-truth target image. Verify it returns a nonzero number.
- [ ] Run Baseline 0 on 3 image pairs. Record the LPIPS scores in the Experiment Log.

Hour 16-32: First Real Model

- [ ] [CONDITIONAL on TA answer] Load InstructPix2Pix from Hugging Face. Run on one image with the prompt 'make this look like a rainy night.'
- [ ] [CONDITIONAL if InstructPix2Pix not allowed] Implement Baseline 1: darken image + add blue-gray tint for rainy night.
- [ ] Run your first model on 5 validation image pairs. Compute LPIPS and condition accuracy (using CLIP zero-shot).
- [ ] Record results in Experiment Log.
- [ ] Push all code to GitHub. Write a README that says how to run the pipeline.

Hour 32-48: Review and Plan

- [ ] Team sync (30 min): review the first results. What looks right? What looks wrong?
- [ ] Write 5 bullet points describing qualitative strengths and weaknesses of the first model output.

- [ ] Plan the next data collection trip. Identify what conditions are missing.
- [ ] Assign Week 2 tasks to each teammate following the timeline in Section 10.
- [ ] Update the metadata CSV with all collected images. Verify no images are missing metadata rows.

> ✅ *After 48 hours you should have: a GitHub repo with code, 30+ labeled images in Drive, a working end-to-end pipeline (however crude), and your first LPIPS score in the Experiment Log. If you have all four of these, you are on track for a strong project.*